K-Means

DSC 340 Applied Machine Learning

Clustering, with *k*-Means as an Example

April 4

The Plan

- Introduce unsupervised learning
- *k*-Means clustering algorithm
- Implementation (see PDSH online)

Supervised vs. Unsupervised Learning

Supervised

- Labeled data
- Classification methods
- Ex.: Logistic regression, *k*NN

Unsupervised

- Data may be unlabeled
- Clustering methods
- Ex.: *k*-Means (today!)
- Can be used for data exploration

k Means

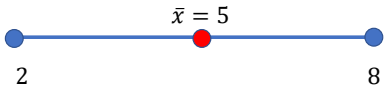
- Suppose we don't have class labels …
- (Labeling data is often expensive!)
- But we want to cluster data into **groups of related items**
- We can choose a **distance metric** (similar to kNN)
- k-Means algorithm finds the **centroids** of $\boldsymbol{k}$ **clusters** of instances in $\mathbf{X}$
  - Centroid: Think "midpoint" but maybe in higher dimensions
  - Average each feature over the instances (points) in the cluster

k Means Algorithm

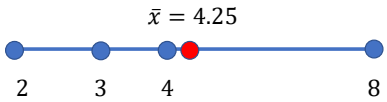
- Ahead of time: Decide on k (**hyperparameter**)
- Step 1: Choose k **random** cluster centers
  - Could choose k random points in min-max **range**
  - Or select k **data points** at random for the **seeds** (without replacement)
- Step 2: Assign each instance to the closest cluster center
  - For example, Euclidean (L2) distance
- Step 3: Recompute cluster centers
  - Compute **centroid** of instances pertaining to each cluster
- Step 4: If any cluster center moved, go to Step 2, else return clusters

Centroid in 1 dimension

- Suppose you have 2 points on a number line



- Suppose you have n points in one dimension (i.e., just one feature)



- Then centroid is just the mean (average)

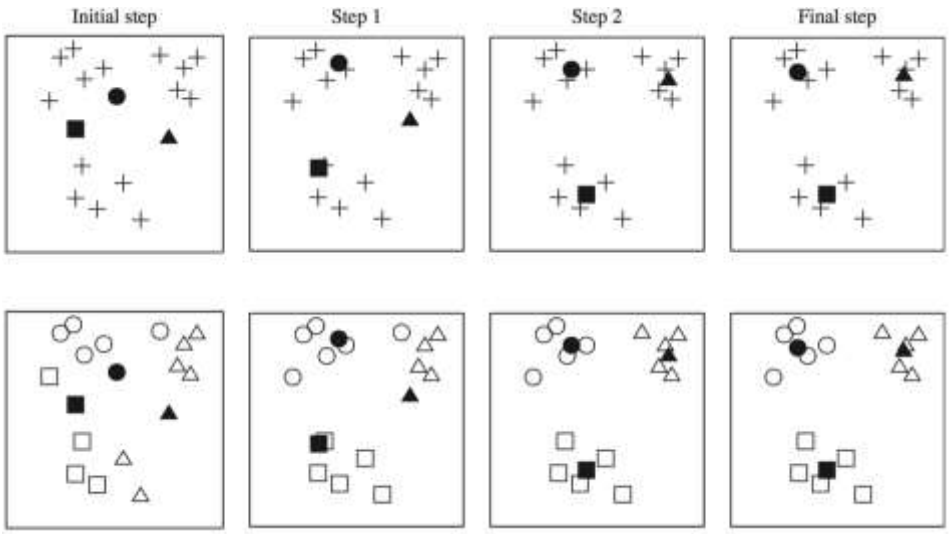
Centroid in m dimensions

- Given n training points (selected rows) …
- with m features (columns) …
- Find **sample mean** (average) of each column for the n points.
- The vector of means is the **centroid**.
  - Note: If we have ***regions*** (shapes), then we need calculus!

What the algorithm does

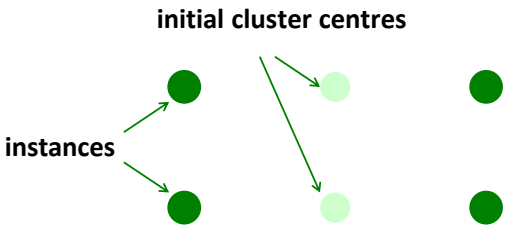
- Guaranteed to terminate
  - Converges when assignments stop changing
  - Or, equivalently, when cluster centers stop changing
  - Finite number of possible assignments
- Minimizes total squared distance …
  - of each point from its assigned cluster center
- Converges to a **local minimum**
  - ⚠ Different initialization can yield different solutions

The *k*-means algorithm: example



Discussion

- Minimizes squared distance to cluster centers
- Result can vary significantly based on initial choice of seeds
- Can get trapped in local minimum
- We can use random restart to improve odds of global optimum



Implementation

- Not hard to implement from scratch
- Alternative:
  - `from sklearn.cluster import KMeans` ☺
  - https://jakevdp.github.io/PythonDataScienceHandbook/05.11-k-means.html

Parting thoughts

- Don't confuse kMeans and kNN!
  - Think: How are they alike? How do they differ?
- What are the parameters? What are the hyperparameters?
  - How find best value of *k*?
- Related methods that achieve faster distance calculations:
  - *k*D-trees and ball trees
  - Hierarchical clustering, cluster into *k* groups then recurse

What are your questions?

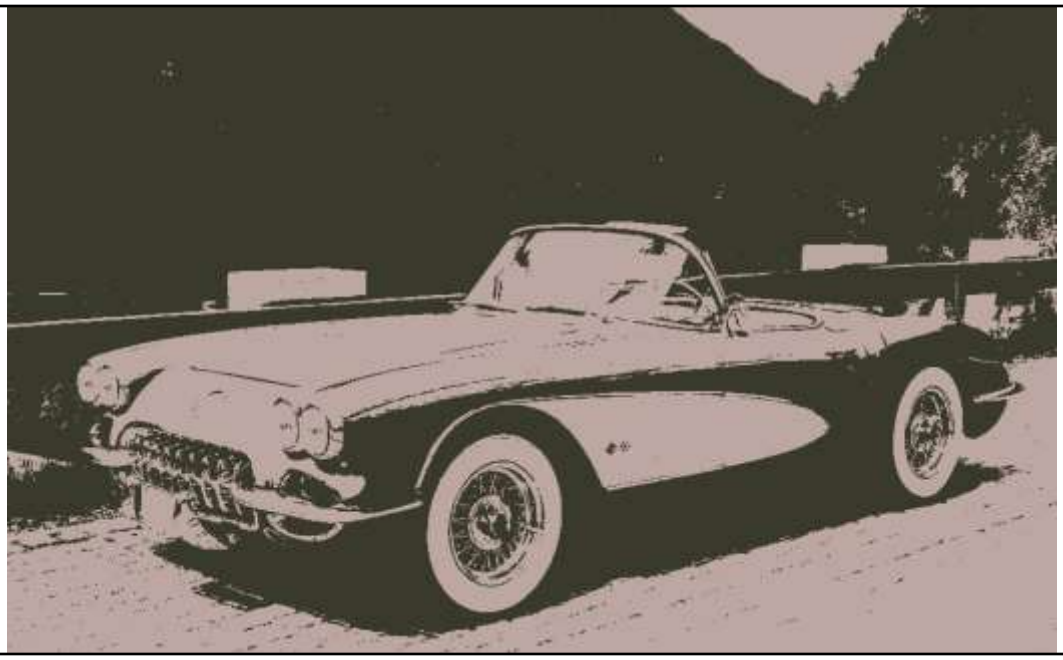
Last Lab!



Original Photo



k=2



k=3



k=5



k=10



